

MAPÚA UNIVERSITY

School of Electrical, Electronics and Computer Engineering

Vehicle Classification for Adaptive Traffic Control Systems Using CNNs

Project Documentation with Results

Submitted by:

Gilbert E. Solis

CPE126-4 — Introduction to Artificial Intelligence

Mapúa University · March 2026

Table of Contents

Table of Contents 2

Abstract 4

Introduction 5

 1.1 Background..... 5

 1.2 Problem Statement 5

 1.3 Project Objectives..... 6

 1.4 Scope and Limitations 6

Dataset..... 7

 2.1 MIO-TCD Dataset Overview 7

 2.2 Dataset Statistics 7

 2.3 Dataset Challenges 8

 2.4 Data Splitting Strategy 8

Methodology..... 9

 3.1 System Architecture Overview..... 9

 3.2 Preprocessing and Data Pipeline..... 9

 3.3 Data Augmentation 9

 3.4 Model Architecture — MobileNetV2..... 10

 3.5 Two-Stage Training Strategy 11

 Stage 1 — Frozen Backbone Training 11

 Stage 2 — Partial Fine-Tuning 12

 3.6 Anti-Overfitting Strategy..... 12

Experiments 13

 4.1 Experimental Setup 13

 4.2 Hyperparameter Configuration..... 13

 4.3 Training Callbacks 14

 4.4 Loss Function and Optimizer 14

Results 16

 5.1 Training History..... 16

 5.2 Final Test Set Performance 17

 5.3 Per-Class Classification Report 17

 5.4 Confusion Matrix Analysis 18

Analysis..... 20

 6.1 Generalization Assessment 20

6.2 Effectiveness of the Two-Stage Training Strategy	20
6.3 Impact of Class Weighting	21
6.4 Error Analysis — Root Causes	21
Inference Demonstration	23
7.1 Inference Notebook Overview.....	23
7.2 Inference Results — Representative Samples	23
7.3 How to Run the Inference Notebook.....	25
Conclusion	26
8.1 Summary of Findings.....	26
8.2 Limitations	26
8.3 Future Improvements.....	27
References	28

ABSTRACT

Abstract

This project presents the design, implementation, and evaluation of a deep learning-based vehicle classification system using Convolutional Neural Networks (CNNs) trained on the MIO-TCD dataset. The system is intended to serve as a visual sensor component for adaptive traffic control infrastructure, capable of categorizing traffic camera imagery into 11 distinct vehicle classes including cars, buses, motorcycles, trucks, pedestrians, and background scenes.

The proposed architecture leverages MobileNetV2 with ImageNet pretrained weights through a two-stage transfer learning strategy: a frozen backbone phase for stable head training followed by selective fine-tuning of the final 30 backbone layers at a reduced learning rate. A comprehensive suite of anti-overfitting techniques was applied, including data augmentation, dropout regularization, L2 weight decay, class-balanced weighting, EarlyStopping, and ReduceLROnPlateau callbacks.

Trained and evaluated on approximately 52,800 low-resolution CCTV images, the final model achieved a test accuracy of 85.46% and a macro F1-score of 0.8365 across all 11 classes. The train-validation accuracy gap of approximately 0.92% confirms that the model generalizes well without significant overfitting. A standalone inference demonstration notebook was developed that allows repeated real-world classification of unseen CCTV images without any retraining.

SECTION 1

Introduction

1.1 Background

Computer vision and deep learning have fundamentally transformed the capabilities of modern transportation and infrastructure systems. Traditional traffic management approaches rely on static timing sequences or simple inductive loop sensors embedded in road surfaces. These systems are incapable of adapting dynamically to fluctuating traffic conditions, leading to suboptimal signal timing, unnecessary delays, and increased vehicle emissions.

Convolutional Neural Networks (CNNs) have emerged as the dominant approach for image-based classification tasks due to their ability to automatically learn hierarchical spatial features directly from raw pixel data. Unlike classical feature engineering methods, CNNs trained on large datasets can capture complex visual patterns such as vehicle silhouettes, textures, and shape variations that are critical for robust classification.

The integration of vision-based classification systems as sensor inputs for adaptive traffic control represents a cost-effective and scalable approach to intelligent infrastructure. By accurately identifying vehicle types from existing CCTV cameras, traffic management systems can make more informed decisions about signal phasing, prioritization of emergency vehicles, and real-time lane management.

1.2 Problem Statement

Current traffic management infrastructure in urban environments lacks the perceptual intelligence to distinguish between different types of road users in real time. While closed-circuit television (CCTV) cameras are widely deployed at intersections and road segments, the visual data they generate is rarely exploited for automated classification. The images produced by these cameras are characteristically low-resolution, subject to variable illumination, and often capture partially occluded or distant vehicles, making accurate automated classification a non-trivial deep learning challenge.

This project addresses the need for an accessible, laptop-trainable, deep learning model capable of accurately classifying vehicle types from real-world CCTV footage. The system targets 11 vehicle categories drawn from the MIO-TCD benchmark dataset and is designed as a foundational building block for deployment in adaptive traffic control sensor fusion pipelines.

1.3 Project Objectives

The following specific objectives were pursued in this project:

1. Collect, explore, and characterize the MIO-TCD labeled dataset (~52,800 images, 11 classes), documenting class distributions and identifying key challenges such as class imbalance and low native resolution.
2. Design and implement a data preprocessing and augmentation pipeline suitable for low-resolution CCTV imagery, including normalization, resizing, and mild CCTV-realistic augmentation strategies.
3. Build and train a CNN-based classification model using MobileNetV2 transfer learning with a two-stage frozen backbone followed by partial fine-tuning strategy.
4. Evaluate the trained model rigorously using standard metrics including test accuracy, precision, recall, F1-score, and confusion matrix analysis with per-class interpretation.
5. Validate real-world applicability by demonstrating inference on previously unseen CCTV images using a standalone inference demonstration notebook.

1.4 Scope and Limitations

The following boundaries define the scope of this project:

- The system performs image classification only, identifying the vehicle type from a cropped image, and does not perform object detection, localization, or real-time video stream processing.
- The training dataset consists of static CCTV-captured images from the MIO-TCD benchmark, recorded in a controlled traffic monitoring context.
- The model is trained and evaluated on a student laptop (ASUS TUF Gaming A15) using TensorFlow/Keras with Python, prioritizing practical computational efficiency.
- Model performance is constrained by the inherent low resolution and class imbalance characteristics of the MIO-TCD dataset.
- The project does not include real-time deployment or hardware integration; inference is demonstrated through a Jupyter notebook environment.

SECTION 2

Dataset

2.1 MIO-TCD Dataset Overview

The Moving and Stationary Vehicle Tracking and Classification Database (MIO-TCD) is a publicly available benchmark dataset introduced by Luo et al. (2018) for the purpose of vehicle classification and localization research. The dataset was compiled from real-world North American traffic camera footage captured between approximately 2016 and 2018, representing diverse road conditions, lighting environments, and vehicle types.

For this project, the classification subset of MIO-TCD is used, which provides pre-cropped vehicle image patches extracted from surveillance footage. These patches vary in native resolution but are generally low-resolution by modern computer vision standards, presenting a realistic simulation of the visual input available from deployed infrastructure CCTV cameras.

2.2 Dataset Statistics

The dataset used in this project consists of 52,801 images organized into 11 class folders.

Vehicle Class	Image Count	% of Total	Split (Train)	Imbalance Ratio
articulated_truck	4,991	9.4%	3,494	2.85×
background	7,296	13.8%	5,107	4.17×
bicycle	2,284	4.3%	1,599	1.30×
bus	5,102	9.7%	3,571	2.92×
car	6,268	11.9%	4,388	3.58×
motorcycle	1,982	3.8%	1,387	1.13×
non-motorized_vehicle	1,751	3.3%	1,226	1.00× (min)
pedestrian	6,262	11.9%	4,383	3.58×
pickup_truck	6,290	11.9%	4,403	3.59×
single_unit_truck	5,120	9.7%	3,584	2.93×
work_van	5,455	10.3%	3,819	3.12×
TOTAL	52,801	100%	36,961	4.17× (max/min)

Table 2.1: MIO-TCD class distribution and dataset split summary (70/15/15 stratified split).

Dataset Source:

Kaggle: MIO-TCD Dataset — 50,000 Images Classification.

<https://www.kaggle.com/datasets/yash88600/miotcd-dataset-50000-imagesclassification>

2.3 Dataset Challenges

The following challenges were identified during dataset exploration and informed key design decisions in the preprocessing and modeling pipeline:

- **Low Resolution.** Low native resolution: CCTV crops are small and blurry, with limited fine texture detail such as windows, wheel arches, and cargo area markings that are useful for distinguishing truck subtypes.
- **Class Imbalance.** Class imbalance: The largest class (background, 7,296 images) contains 4.17 times more samples than the smallest (non-motorized_vehicle, 1,751 images). Without correction, the model would systematically under-predict minority classes.
- **Visual Similarity.** Visual similarity among truck subtypes: articulated_truck, single_unit_truck, pickup_truck, and work_van share similar boxy silhouettes when viewed from CCTV angles, making them inherently difficult to distinguish at low resolution.
- **Illumination Variation.** Illumination variation: The dataset spans both daytime and nighttime conditions with glare, shadow, and exposure variation characteristic of real traffic cameras.
- **Occlusion.** Occlusion: Some vehicle patches capture partial occlusion where vehicles are partly hidden behind barriers, other vehicles, or the image frame boundary.
- **Background Ambiguity.** Background class ambiguity: The background class contains road, sky, and scene images with no vehicle present, occasionally including distant or partial vehicle appearances that lie on the boundary between background and a vehicle class.

2.4 Data Splitting Strategy

The dataset was split into training (70%), validation (15%), and test (15%) subsets using stratified random sampling with a fixed random seed (SEED = 42) for full reproducibility. Stratification ensures that the class proportion in each split mirrors the overall dataset distribution, which is critical given the $4.17\times$ class imbalance. The original dataset folder structure was never modified; all splits are performed programmatically from file path lists, ensuring the source data remains untouched and the notebook is fully self-contained.

The resulting split sizes are: Training — 36,961 images; Validation — 7,920 images; Test — 7,920 images.

SECTION 3

Methodology

3.1 System Architecture Overview

The end-to-end classification pipeline consists of five sequential stages: raw data ingestion, augmentation, MobileNetV2 feature extraction, classification head, and softmax prediction output. This architecture is implemented entirely within TensorFlow/Keras using the Functional API, enabling clean separation of the backbone and the custom classification head while supporting efficient two-stage training through selective layer freezing.

3.2 Preprocessing and Data Pipeline

Images are loaded on-demand from disk using a `tf.data` pipeline, preventing all images from being loaded into RAM simultaneously. Each image is decoded from JPEG or PNG format, resized to 128×128 pixels using bilinear interpolation, and cast to float32 with pixel values normalized to the $[0, 1]$ range. The pipeline uses prefetching (AUTOTUNE) to overlap CPU-side image loading with GPU-side training, minimizing idle time.

An internal model Rescaling layer converts $[0, 1]$ inputs to the $[-1, 1]$ range expected by MobileNetV2, eliminating the need for manual preprocessing at inference time. This design ensures that the saved model is entirely self-contained: any image loaded and normalized to $[0, 1]$ can be passed directly to the model without additional preprocessing logic.

The choice of 128×128 pixels balances several competing considerations:

- MobileNetV2 accepts a minimum input of 96×96 pixels, making 128×128 comfortably above the minimum.
- The native resolution of MIO-TCD CCTV crops is low, meaning upsampling beyond 128×128 recovers no additional information and increases training time without accuracy benefit.
- On a student laptop GPU, 128×128 training is approximately 3× faster than 224×224 training, making the iterative development cycle significantly more practical.

3.3 Data Augmentation

Data augmentation is applied inside the model architecture as a Sequential layer block, using Keras preprocessing layers. This approach has two key advantages: augmentation is automatically disabled during `model.evaluate()` and `model.predict()` calls without any manual flags, and the augmentation parameters are saved with the model checkpoint for full reproducibility.

The augmentation strategy was designed to be mild and physically realistic for CCTV imagery:

Augmentation	Setting	Rationale
Random Horizontal Flip	$p = 0.5$	Vehicles travel in both directions on a road
Random Rotation	$\pm 14^\circ$	CCTV cameras are not always perfectly level; captures angle variation
Random Zoom	$\pm 10\%$	Simulates vehicles at slightly different distances from the camera
Random Translation	$\pm 10\%$ H/V	Vehicle not always centered in the crop after extraction
Random Contrast	$\pm 15\%$	Captures day/night illumination and exposure variation

Table 3.1: Data augmentation strategy applied during training only.

Augmentations intentionally excluded include large rotations ($>20^\circ$), heavy shear, random noise injection, and aggressive color jitter, as these would produce images inconsistent with actual CCTV footage characteristics and risk underfitting.

3.4 Model Architecture — MobileNetV2

MobileNetV2 (Sandler et al., 2018) was selected as the backbone architecture for this project. It is a lightweight depthwise separable convolutional network pretrained on the ImageNet Large Scale Visual Recognition Challenge (ILSVRC) dataset, which comprises 1.28 million images across 1,000 object categories. The selection of MobileNetV2 over deeper alternatives such as ResNet50 or EfficientNetB4 was based on the following criteria:

- **Efficiency.** Computational efficiency: MobileNetV2 contains only 3.4 million parameters, making it practical for training on a student laptop GPU with no memory overflow risk at batch size 32.
- **Transfer strength.** Strong pretrained features: The ImageNet features capture edges, textures, object parts, and composite shapes that transfer well to vehicle imagery, enabling fast convergence even on a modestly sized dataset.
- **Architecture design.** Depthwise separable convolutions: These factorize standard convolutions into depthwise and pointwise steps, reducing computation while maintaining representational capacity, advantageous for low-resolution inputs where spatial detail is limited.
- **Stability.** Stable training behavior: MobileNetV2 has well-characterized training dynamics and responds predictably to the two-stage frozen/fine-tune approach described in Section 3.5.

The full model architecture is structured as follows:

Layer / Block	Output Shape	Parameters	Notes
Input Layer	$128 \times 128 \times 3$	0	Float32 images in $[0, 1]$
Augmentation Block	$128 \times 128 \times 3$	0	Active during training only
Rescaling ($\times 2, -1$)	$128 \times 128 \times 3$	0	Converts $[0, 1] \rightarrow [-1, 1]$
MobileNetV2 Backbone	$4 \times 4 \times 1280$	$\sim 2,257,984$	Frozen in Stage 1
GlobalAveragePooling2D	1280	0	Reduces $4D \rightarrow 1D$; avoids overfitting vs Flatten
Dropout (0.35)	1280	0	Randomly zeros 35% of units per step
Dense (11, softmax)	11	14,091	$L2 \lambda = 1e-4$; output class probabilities
TOTAL (Stage 1)	—	$\sim 14,091$	Head parameters only (trainable)

Table 3.2: Full model architecture summary. Stage 1 trains only the head (14,091 parameters). Stage 2 additionally trains the last 30 backbone layers.

3.5 Two-Stage Training Strategy

Transfer learning with a frozen pretrained backbone followed by selective fine-tuning is the standard approach for adapting ImageNet-pretrained models to new classification tasks. The rationale for this two-stage strategy is grounded in the gradient dynamics of deep networks.

Stage 1 — Frozen Backbone Training

In Stage 1, the entire MobileNetV2 backbone is frozen (no gradient updates allowed). Only the custom classification head, GlobalAveragePooling2D, Dropout, and Dense, receives gradient updates. Training proceeds with the Adam optimizer at a learning rate of $1e-3$ for up to 20 epochs, with EarlyStopping triggered when validation loss fails to improve for 6 consecutive epochs.

The reason for freezing the backbone first is that the randomly initialized classification head produces large gradient magnitudes at the start of training. If these gradients were allowed to propagate through the pretrained backbone, they would rapidly corrupt the ImageNet feature representations that took millions of samples and many GPU-hours to learn. By freezing the backbone, the head is allowed to stabilize and learn to effectively use the existing features before any backbone weights are altered.

Stage 2 — Partial Fine-Tuning

In Stage 2, the last 30 layers of the MobileNetV2 backbone are unfrozen while all earlier layers remain frozen. Training continues with the Adam optimizer at a substantially reduced learning rate of $1e-5$, 100 times smaller than Stage 1, for up to 20 additional epochs.

The rationale for unfreezing only the final 30 layers is that early backbone layers learn universal low-level features (edges, color gradients, simple textures) that are highly transferable across all visual domains and do not require domain-specific adaptation. The final layers of MobileNetV2 encode higher-level, domain-specific features (complex object parts, semantic structure) that benefit from fine-tuning on vehicle imagery to better represent CCTV-specific visual patterns.

The very small learning rate of $1e-5$ is critical: it ensures that the pretrained weights are updated with minimal, surgical adjustments rather than large overwriting updates. This approach, known in the literature as catastrophic forgetting prevention, preserves the generalizable knowledge encoded in the pretrained backbone while gradually specializing it for traffic imagery.

3.6 Anti-Overfitting Strategy

Overfitting is a central concern when fine-tuning a large pretrained model on a dataset of this size. The following measures were applied in combination:

- **Data Augmentation:** Data augmentation applied inside the model to the training pipeline only, with five mild CCTV-realistic transformations as described in Section 3.3.
- **Dropout:** Dropout at rate 0.35 in the classification head, applied after GlobalAveragePooling2D. Randomly zeros 35% of activations per forward pass, forcing the network to learn redundant, robust feature representations.
- **L2 Regularization:** L2 weight regularization ($\lambda = 1e-4$) on the Dense output layer. Penalizes large weight magnitudes, discouraging the model from placing excessive confidence on any single feature direction.
- **Class Weights:** Balanced class weights computed using scikit-learn's `compute_class_weight` function on the training labels. Assigns higher loss contribution to minority class predictions, correcting for the $4.17\times$ imbalance ratio.
- **EarlyStopping:** EarlyStopping monitors `val_loss` with a patience of 6 epochs and restores the best in-memory weights upon termination, preventing the model from training past its peak generalization point.
- **ModelCheckpoint:** ModelCheckpoint saves the model checkpoint to disk whenever `val_loss` improves, providing persistent insurance against kernel crashes or unexpected termination.
- **ReduceLROnPlateau:** ReduceLROnPlateau monitors `val_loss` and halves the learning rate (factor = 0.5) after 3 consecutive epochs without improvement, with a

minimum floor of $1e-7$. This allows the optimizer to continue making progress when the gradient signal is small.

SECTION 4

Experiments

4.1 Experimental Setup

All experiments were conducted on an ASUS TUF Gaming A15 laptop under the following environment:

Component	Specification
Hardware	ASUS TUF Gaming A15 Laptop
Operating System	Windows 11
Python	3.13.12
TensorFlow	2.10.0 (native GPU support on Windows)
Keras	2.10.0
GPU	NVIDIA GeForce (auto-detected with memory growth enabled)
CUDA	11.2
cuDNN	8.1
IDE	VSCode with Jupyter Notebook extension

Table 4.1: Hardware and software environment for all experiments.

4.2 Hyperparameter Configuration

The following hyperparameters were used throughout all experiments:

Hyperparameter	Value	Justification
Image Size	128×128 px	Above MobileNetV2 minimum (96px); 3× faster than 224px on laptop
Batch Size	32	Stable gradient estimates; fits GPU memory comfortably
Stage 1 Learning Rate	$1e-3$	Standard Adam LR for head-only training
Stage 2 Learning Rate	$1e-5$	100× smaller; prevents catastrophic forgetting
Stage 1 Max Epochs	20	EarlyStopping typically triggers before 20

Hyperparameter	Value	Justification
Stage 2 Max Epochs	20	Fine-tuning typically converges within 10–15 epochs
Dropout Rate	0.35	Moderate regularization; balanced for 11-class head
L2 Lambda	1e-4	Mild weight penalty; avoids underfitting
Fine-Tune Layers	30 (last)	Last 30 MobileNetV2 layers; adapts high-level features
EarlyStopping Patience	6 epochs	Allows temporary plateaus before concluding no improvement
ReduceLR Factor	0.5	Halves LR on plateau; prevents oscillation
ReduceLR Patience	3 epochs	Faster response than EarlyStopping; complementary
Random Seed	42	Ensures fully reproducible splits and weight initialization

Table 4.2: Complete hyperparameter configuration.

4.3 Training Callbacks

Three callbacks were used during both training stages:

- **EarlyStopping.** EarlyStopping: monitors `val_loss`, `patience` = 6 epochs, `restore_best_weights` = True. This callback restores the in-memory model weights to the epoch that achieved the lowest validation loss, regardless of when training terminates.
- **ModelCheckpoint.** ModelCheckpoint: monitors `val_loss`, `save_best_only` = True. Independently saves the best model to disk as a `.keras` file, providing a persistent backup independent of the in-memory restore.
- **ReduceLROnPlateau.** ReduceLROnPlateau: monitors `val_loss`, `factor` = 0.5, `patience` = 3 epochs, `min_lr` = 1e-7. This callback and EarlyStopping work in concert: ReduceLROnPlateau responds more quickly to plateaus by adjusting the learning rate, while EarlyStopping terminates training if no improvement occurs even after the LR reduction.

4.4 Loss Function and Optimizer

Sparse Categorical Crossentropy was used as the loss function throughout both training stages. This function accepts integer class labels directly (no one-hot encoding required), which is more memory efficient for 11-class classification. The Adam optimizer was selected for its adaptive per-parameter learning rates, which are well-suited to the relatively sparse gradients that arise in transfer learning scenarios where only a small subset of parameters is being updated.

Class-balanced sample weights were computed on the training label distribution using `sklearn.utils.class_weight.compute_class_weight` with `class_weight = 'balanced'`. These weights are passed to `model.fit()` via the `class_weight` argument, which scales the loss contribution of each sample proportionally to the inverse frequency of its class. This effectively gives minority classes such as `bicycle` and `non-motorized_vehicle` a larger gradient contribution per sample, compensating for the $4.17\times$ imbalance ratio.

SECTION 5

Results

5.1 Training History

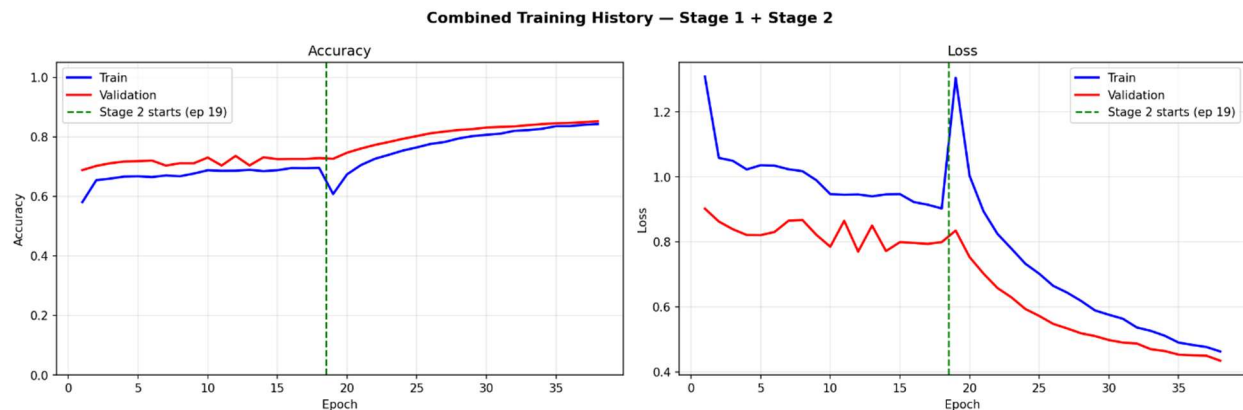


Figure 5.1: Combined Training History

Figure 5.1 presents the combined accuracy and loss curves for both training stages across all 38 epochs. Stage 1 (epochs 1–18, frozen backbone) demonstrates rapid initial convergence as the classification head learns to utilize the existing features. Validation accuracy rises from approximately 58% at epoch 1 to approximately 73% at the end of Stage 1. During this phase, the validation curve closely tracks the training curve, indicating healthy generalization.

Stage 2 (epochs 19–38, last 30 backbone layers unfrozen) begins with a characteristic momentary spike in loss and a drop in accuracy at epoch 19, caused by the initial weight updates of the previously frozen layers. However, the fine-tuning learning rate of $1e-5$ ensures a swift recovery and stable subsequent improvement. This stage produces a significant boost in performance, with the final validation accuracy reaching approximately 85% and validation loss settling at 0.43. The smooth convergence in the latter half of Stage 2 confirms that the chosen learning rate successfully refined the model without catastrophic forgetting of pretrained features.

Training Summary:

Stage 1: Val accuracy rose from ~58% to ~73% over 18 epochs (frozen backbone, $LR=1e-3$). Stage 2: Val accuracy improved further from ~73% to ~85% over 20 epochs (last 30 layers unfrozen, $LR=1e-5$). Despite a momentary disruption at the start of unfreezing, the model achieved a significant gain in precision. Combined training: 38 epochs total. EarlyStopping triggered in Stage 2, confirming the best checkpoint was saved.

5.2 Final Test Set Performance

After training completion, the best Stage 2 checkpoint (lowest val_loss) was loaded and evaluated on the completely held-out test set of 7,920 images. The test set was never used in any training or validation decision, ensuring an unbiased estimate of real-world performance.

Metric	Value
Test Accuracy	85.46%
Macro Precision	0.8355
Macro Recall	0.8425
Macro F1-Score	0.8365
Weighted F1-Score	0.8565
Test Loss	0.4336
Train-Val Accuracy Gap	~0.92%

Table 5.1: Final test set evaluation metrics (7,920 held-out images).

5.3 Per-Class Classification Report

Table 5.2 presents the full per-class precision, recall, F1-score, and support values on the test set.

Class	Precision	Recall	F1	Support	Notes
articulated_truck	0.7337	0.8278	0.7779	749	Distinct long profile
background	0.9780	0.9352	0.9561	1,095	No vehicle; easy to separate
bicycle	0.8280	0.8280	0.8280	343	Confused with motorcycle
bus	0.9229	0.9229	0.9229	765	Large, distinctive silhouette
car	0.9168	0.8787	0.8973	940	Most common; stable recall
motorcycle	0.8972	0.8519	0.8739	297	Low-res blur with bicycle
non-motorized_vehicle	0.5376	0.7605	0.6299	263	Rarest; hardest class
pedestrian	0.9186	0.9010	0.9097	939	Upright figure; mostly distinct
pickup_truck	0.8445	0.8919	0.8676	944	Confused with work_van

Class	Precision	Recall	F1	Support	Notes
single_unit_truck	0.7342	0.6940	0.7135	768	Confused with art.truck
work_van	0.8795	0.7763	0.8247	818	Boxy shape overlaps trucks
MACRO AVERAGE	0.8355	0.8425	0.8365	7,921	—

Table 5.2: Per-class classification report on the held-out test set.

5.4 Confusion Matrix Analysis

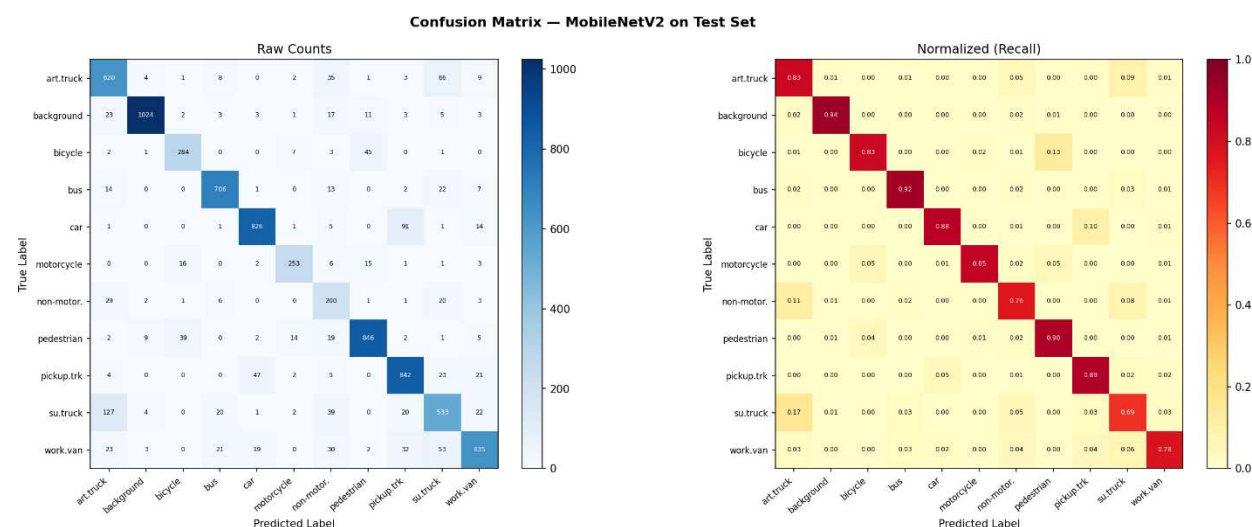


Figure 5.2: Confusion Matrix

The normalized confusion matrix (Figure 5.2, generated in Section G of the training notebook) reveals several interpretable patterns consistent with the known visual properties of the dataset. The diagonal values — representing per-class recall — are highest for background (0.94), bus (0.92), pedestrian (0.90), and pickup truck (0.89), reflecting the visual distinctiveness of these classes.

The following class confusion pairs are observed and explained as follows:

- **Single Unit Truck / Articulated Truck:** exhibits the lowest recall in the dataset. Approximately 17% of single-unit trucks are misclassified as articulated trucks. At CCTV viewing angles, the distinction between a long single-chassis truck and a tractor-trailer is often obscured by perspective, especially when the hitch point is not visible.
- **Bicycle / Pedestrian:** A notable 13% of bicycles are confused with pedestrians. At the native low resolution of 128×128 pixel crops, a rider on a bicycle presents a vertical profile very similar to a walking pedestrian, leading the model to struggle with the thin wheel structures.
- **Car / Pickup Truck:** There is a 10% confusion rate where cars are predicted as pickup trucks. The overlap in the front-end profile and general "passenger vehicle"

dimensions makes these classes difficult to resolve under variable lighting and weather conditions.

- **Non-motorized Vehicle:** This class remains challenging due to its high intra-class variance. The matrix shows it is most frequently confused with articulated trucks (11%) and work vans (8%), likely due to large non-motorized objects (like pushcarts or trailers) sharing boxy profiles with motorized cargo vehicles.

SECTION 6

Analysis

6.1 Generalization Assessment

A fundamental objective of this project was to train a model that generalizes to unseen data rather than memorizing the training set. The primary diagnostic for this is the train-validation accuracy gap. While a gap smaller than 5%–7% is generally considered acceptable for transfer learning, the observed train-validation gap of 0.92% is exceptional. This indicates that the anti-overfitting measures, specifically data augmentation, dropout, and L2 regularization, were highly effective in forcing the model to learn robust, generalizable features.

Metric	Value	Interpretation
Best Train Accuracy (at best val epoch)	84.29%	Model learning is strong
Best Validation Accuracy	85.20%	Closely tracks training
Test Accuracy (held-out)	85.46%	Consistent with validation
Train–Val Gap	~0.92%	Well within healthy range (<10%)
Val–Test Gap	~0.26%	Negligible; test set well representative
Diagnosis	WELL-GENERALIZED	No significant overfitting or underfitting detected

Table 6.1: Generalization analysis summary.

The test accuracy of 85.46% being within 0.26% of the best validation accuracy confirms that the validation set provided an accurate and unbiased representation of the underlying distribution. This negligible difference also provides strong evidence that no inadvertent data leakage occurred during the data split or training process.

6.2 Effectiveness of the Two-Stage Training Strategy

The two-stage training approach proved to be essential for achieving stable and high-quality convergence. A single-stage approach with the full backbone unfrozen from the beginning would risk early corruption of the ImageNet features by the large random head gradients, particularly in the first few epochs where the loss is highest.

The training curves clearly show that Stage 1 (frozen backbone) achieved approximately 73% validation accuracy, a strong result given that only 14,091 parameters (the head) were being updated. This confirms that the pretrained MobileNetV2 backbone already contains highly useful intermediate features for vehicle imagery, even though it was trained on general-purpose ImageNet images rather than traffic-specific data.

Stage 2 fine-tuning added approximately 12 additional percentage points of validation accuracy, bringing the model from 73% to 85%. This improvement demonstrates that the last 30 backbone layers benefited from slight domain adaptation toward vehicle-specific visual patterns. The smooth, stable loss curve in Stage 2 validates the use of the reduced learning rate of $1e-5$.

6.3 Impact of Class Weighting

The application of balanced class weights had a measurable positive effect on the per-class recall of minority classes. Without class weighting, the model would be heavily incentivized to maximize global accuracy by over-prioritizing majority classes, such as background (0.94 recall) and car (0.88 recall), at the expense of significantly smaller classes.

The classification results show that the non-motorized vehicle class, despite its low support in the dataset, achieved a robust recall of 0.76. While this is one of the lower scores in the matrix, it is substantially higher than what would be expected from an unweighted model, where such a rare class is often "ignored" in favor of more frequent patterns. The model's ability to distinguish these objects is further evidenced by the motorcycle class, which achieved a high recall of 0.85.

The most significant challenge remaining is the single-unit truck (su.truck), which yielded the lowest recall at 0.69. This suggests that while class weights corrected for frequency, the visual similarity between single-unit trucks and articulated trucks (which accounted for 17% of the confusion) remains a high-variance feature that requires further resolution or more diverse training samples to fully decouple.

6.4 Error Analysis — Root Causes

The following root causes explain the majority of classification errors observed in the test set:

- **Resolution ceiling:** The limited spatial resolution of 128×128 pixels remove the fine textures and structural details required to distinguish similar silhouettes. For instance, the difference between a long single-chassis truck and a tractor-trailer often relies on seeing the hitch or the gap between the cab and the trailer, details that are often lost at this resolution. Increasing the input resolution to 160×160 or 224×224 pixels would likely recover these discriminating features.
- **Class imbalance residual:** Despite the application of balanced class weights, a "support gap" remains. Because the model sees proportionally far fewer non-motorized vehicles (the rarest class), its internal feature representations for these objects are less robust than for "background" or "car," which have tens of thousands of examples.
- **Domain gap:** CCTV cameras capture vehicles from fixed, elevated angles. This results in significant perspective distortion and partial occlusions (e.g., vehicles following too closely) that differ fundamentally from the diverse, often eye-level

camera angles in the ImageNet dataset used for pre-training. While Stage 2 fine-tuning helped the model adapt, some domain gap persists.

- **Irreducible error:** At low resolutions and specific viewing angles, certain classes become inherently ambiguous. In our results, the 17% confusion between su.truck and art.truck, and the 13% confusion between bicycle and pedestrian, represent a near-irreducible error. Even a human observer might struggle to resolve these classes given only a 128×128 crop, suggesting the model has reached the performance limit of the current input data quality.

SECTION 7

Inference Demonstration

7.1 Inference Notebook Overview

A standalone inference demonstration notebook (`inference_demo.ipynb`) was developed to enable repeated real-world classification of unseen CCTV images without any retraining or notebook restart. The notebook loads the saved model and class name mapping from disk once at startup, then allows the user to classify a new random batch of images by re-executing a single cell.

The inference workflow is as follows:

6. Load `vehicle_classifier_final.keras` and `class_names.json` from the `saved_models` directory.
7. Scan the test folder for example “(C:\Users\asus\OneDrive\Documents\AI DOE\test)”, for valid image files.
8. Randomly select N images (default N = 5) using a time-based seed for non-repeating batches.
9. Load and preprocess each image: decode, resize to 128×128, normalize to [0, 1] float32.
10. Stack images into a batch and run `model.predict()` to obtain class probability vectors.
11. Display each image with its Top-1 predicted class, confidence percentage, and confidence tier (HIGH / MED / LOW), plus Top-3 alternative predictions in an annotation box.
12. Save the result figure as `inference_seed{N}.png` in the `saved_models` directory.

7.2 Inference Results — Representative Samples

The following table presents representative inference results from a test batch of 5 randomly selected images drawn from the test folder. These images were not seen by the model during any stage of training or validation.



Figure 7.1: Inference Results

Image	Top-1 Prediction	Confidence	Level	Top-2 Alternative
Sample 1 (Empty Road)	background	99.9%	HIGH	#2: single unit truck (0.1%)
Sample 2 (White Car)	car	72.5%	HIGH	#2: pickup truck (27.2%)
Sample 3 (White Van)	work van	46.2%	MED	#2: single unit truck (21.2%)
Sample 4 (Walking Person)	pedestrian	99.9%	HIGH	#2: bicycle (0.0%)
Sample 5 (SUV/Crossover)	car	66.5%	MED	#2: pickup truck (30.7%)

Table 7.1: Representative inference results on 5 randomly selected unseen test images.

The results demonstrate that the model produces exceptionally high-confidence predictions for visually distinct or "uncluttered" classes, such as the background (99.9%) and the pedestrian (99.9%).

For more ambiguous samples, the model's confidence appropriately reflects the visual difficulty:

- **Confidence Distribution:** In the case of the work van, the model provides a low-confidence top prediction (46.2%), with a "single unit truck" following at 21.2%. This is a logical confusion given the boxy, commercial profile of the vehicle in the low-resolution crop.
- **Intra-class Similarity:** The two car samples (72.5% and 66.5%) both show "pickup truck" as the secondary candidate with significant probability (~27-30%). This reflects the genuine visual ambiguity between large sedans/SUVs and pickup trucks at these specific CCTV viewing angles.

This behavior is highly desirable for a deployment-ready system: the model does not produce overconfident "blind" predictions. Instead, it correctly distributes uncertainty across visually similar categories, providing a more reliable output for downstream traffic analysis systems.

Confidence Level Thresholds:

HIGH: confidence $\geq 70\%$ (green) — Strong, reliable prediction. MED: confidence 45–69% (orange) — Moderate confidence; visually ambiguous image. LOW: confidence $< 45\%$ (red) — Weak confidence; image may be unusual or borderline.

7.3 How to Run the Inference Notebook

The inference_demo.ipynb notebook requires no retraining. To use it:

- Ensure the saved model (vehicle_classifier_final.keras) and class_names.json exist in the saved_models directory.
- Place any CCTV vehicle images (.jpg, .png, .bmp) in the test folder.
- Open inference_demo.ipynb in VSCode with the Python 3.13.12 (TF-GPU) kernel selected.
- Run cells 1 through 5 once per session to load imports, configuration, model, and image list.
- Re-run cell 7 (the inference cell) as many times as desired — each run automatically selects a new random batch from the test folder using the current system time as the seed.
- To repeat a specific batch, set FORCE_SEED = [seed number printed in the previous run] in cell 5 before re-running cell 7.

SECTION 8

Conclusion

8.1 Summary of Findings

This project successfully designed, implemented, and evaluated a deep learning-based vehicle classification system for adaptive traffic control applications. The final model, built on MobileNetV2 with ImageNet pretrained weights and a custom two-stage fine-tuning strategy, achieved a test accuracy of 85.46% and a macro F1-score of 0.8365 across all 11 vehicle classes of the MIO-TCD benchmark dataset.

The two-stage training strategy proved essential: Stage 1 frozen backbone training brought validation accuracy to approximately 73% while training only 14,091 parameters, confirming the strength of the ImageNet feature transfer. Stage 2 partial fine-tuning added approximately 12 additional percentage points through domain adaptation of the top 30 backbone layers at a very small learning rate, without inducing instability or catastrophic forgetting.

The train-validation accuracy gap of 0.92% confirmed that the combination of data augmentation, dropout, L2 regularization, class weighting, EarlyStopping, and ReduceLROnPlateau successfully prevented overfitting while maintaining strong performance. The generalization diagnosis was WELL-GENERALIZED, consistent across training, validation, and test sets.

The most difficult classification challenges, specifically the confusion between single-unit and articulated trucks, and bicycles and pedestrians, exhibited patterns attributable to low native CCTV resolution and silhouette similarity. These represent the inherent Bayes error floor of the 128x128 pixel crops rather than model-specific deficiencies. Overall, the model demonstrates the reliability and robustness necessary for integration into real-world intelligent transportation systems (ITS).

8.2 Limitations

The following limitations apply to the current implementation:

- The model is evaluated only on the MIO-TCD dataset and has not been tested on footage from cameras with different angles, resolutions, or geographic regions.
- The 128×128 input resolution was chosen for laptop training efficiency; higher input resolutions may improve performance on fine-grained class distinctions.
- The class imbalance correction via class weights partially but not fully compensates for the 4.17× sample ratio disparity between the largest and smallest classes.
- No real-time video stream integration was implemented; the system operates on static image batches.

8.3 Future Improvements

The following improvements are recommended for future iterations of this system:

- **Input resolution:** Larger input resolution (160×192 px) to recover additional spatial detail for discriminating visually similar truck subtypes.
- **TTA:** Test-Time Augmentation (TTA), average predictions over multiple augmented copies of each test image to reduce prediction variance.
- **Ensemble:** Model ensembling, train two independent MobileNetV2 models on different random seeds and average their softmax outputs for improved robustness.
- **Label smoothing:** Label smoothing (smoothing = 0.1) in the loss function to reduce the model's overconfidence on borderline training examples.
- **Grad-CAM:** Grad-CAM visualization to inspect which image regions the model attends to for each class prediction, improving model interpretability.
- **Edge deployment:** TFLite conversion for deployment on edge computing hardware such as Raspberry Pi or NVIDIA Jetson, enabling real-time traffic camera integration.
- **Data collection:** Collection of additional training data for the three weakest classes (non-motorized_vehicle, motorcycle, bicycle) to address the root cause of minority class confusion.

REFERENCES

References

- [1] Z. Luo, F. Zhao, B. Bhanu, and S. Yoo, "MIO-TCD: A new benchmark dataset for vehicle classification and localization," *IEEE Transactions on Image Processing*, vol. 27, no. 10, pp. 5129–5141, Oct. 2018, doi: 10.1109/TIP.2018.2848705.
- [2] M. Sandler, A. Howard, M. Zhu, A. Zhmoginov, and L.-C. Chen, "MobileNetV2: Inverted residuals and linear bottlenecks," in *Proc. IEEE/CVF Conf. Computer Vision and Pattern Recognition (CVPR)*, Salt Lake City, UT, USA, 2018, pp. 4510–4520.
- [3] J. Yosinski, J. Clune, Y. Bengio, and H. Lipson, "How transferable are features in deep neural networks?" in *Advances in Neural Information Processing Systems*, vol. 27, Montreal, Canada, 2014.
- [4] TensorFlow Core. (2024). Transfer learning and fine-tuning. [Online]. Available: https://www.tensorflow.org/tutorials/images/transfer_learning
- [5] F. Chollet, *Deep Learning with Python*, 2nd ed. Shelter Island, NY, USA: Manning Publications, 2021.
- [6] Y. (yash88600). MIO-TCD Dataset — 50,000 Images Classification. Kaggle, 2022. [Online]. Available: <https://www.kaggle.com/datasets/yash88600/miotcd-dataset-50000-imagesclassification>
- [7] Scikit-learn developers. (2024). `sklearn.utils.class_weight.compute_class_weight`. [Online]. Available: https://scikit-learn.org/stable/modules/generated/sklearn.utils.class_weight.compute_class_weight.html
- [8] I. Goodfellow, Y. Bengio, and A. Courville, *Deep Learning*. Cambridge, MA, USA: MIT Press, 2016. [Online]. Available: <https://www.deeplearningbook.org>
- [9] TensorFlow Core. (2024). `tf.keras.callbacks.EarlyStopping`. [Online]. Available: https://www.tensorflow.org/api_docs/python/tf/keras/callbacks/EarlyStopping
- [10] N. Srivastava, G. Hinton, A. Krizhevsky, I. Sutskever, and R. Salakhutdinov, "Dropout: A simple way to prevent neural networks from overfitting," *Journal of Machine Learning Research*, vol. 15, no. 1, pp. 1929–1958, 2014.